Penetration Report

Testers: Krishanth Babu and Yat Wang

Overview

This is our penetration testing report, which displays the results of a security test based around our Secure Document Sharing program built with Flask. In order to test our application, we used OWASP's Top 10 vulnerabilities, and our own project specific security requirements. Our main focuses revolved around authentication, authorization, input validation, session management, and system configuration.

Overall, our system held up nicely against all security challenges thrown at it. We were able to find 0 critical security breaches or errors, with all required security measures implemented throughout the code.

Testing Methodology

Categories	**Tests**
Authentication	Brute force, Password Complexity, Session Management, Logout Functionality, Password Reset
Authorization	Horizontal/Vertical escalation, Direct object reference, Forced Browsing
Input Validation	XSS, Path Traversal, Command Injection, File Upload
Session Security	Session Fixation, Hijacking, Concurrent Sessions, Session Timeout
Configuration	Security Headers, TLS/SSL, Error Handling, Debug Mode

Testing Tools: Manual testing, Browser Developer Tools, and SecurityHeaders.com

Testing Accounts:
- Admin
- Test1 (user)
- Test2 (user)

- Test3 (user)
- Guest

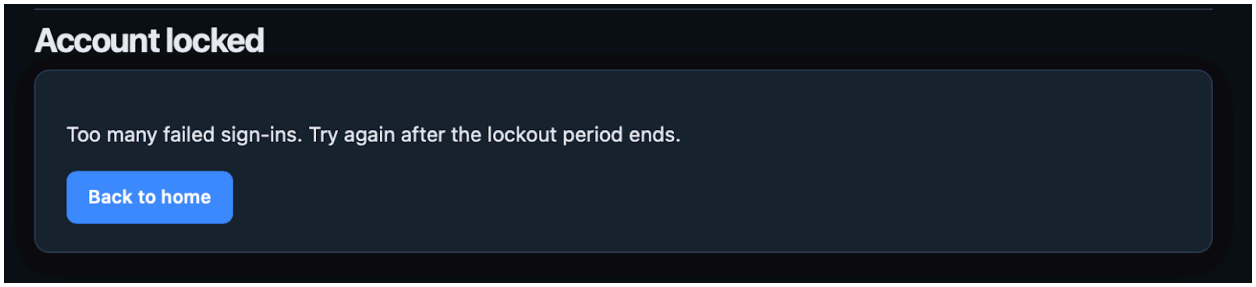
Listed below is every individual test that occurred during our Penetration Report. You will see the final result of the test, and a short summary of what we did to get to that point alongside any findings noteworthy.

Findings

A. Authentication Testing (10 points)

- Brute Force Protection

Invalid Credentials (Tested with entering the wrong password 5 times in a row to a registered user)



LOGIN_BLOCKED_LOCKED | user=test2 | ip=127.0.0.1 | details=- | ua=Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/605.1.15 (KHTML, like Gecko) Version/26.3.1 Safari/605.1.15 | ts=2026-04-19 10:28:53

LOGIN_ATTEMPT | user=test2 | ip=127.0.0.1 | details=- | ua=Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/605.1.15 (KHTML, like Gecko) Version/26.3.1 Safari/605.1.15 | ts=2026-04-19 10:28:53

ACCOUNT_LOCKED | user=test2 | ip=127.0.0.1 | details=failed_attempts_exceeded | ua=Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/605.1.15 (KHTML, like Gecko) Version/26.3.1 Safari/605.1.15 | ts=2026-04-19 10:28:49

LOGIN_FAILED | user=test2 | ip=127.0.0.1 | details=invalid_credentials | ua=Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/605.1.15 (KHTML, like Gecko) Version/26.3.1 Safari/605.1.15 | ts=2026-04-19 10:28:49

LOGIN_ATTEMPT | user=test2 | ip=127.0.0.1 | details=- | ua=Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/605.1.15 (KHTML, like Gecko) Version/26.3.1 Safari/605.1.15 | ts=2026-04-19 10:28:49

LOGIN_FAILED | user=test2 | ip=127.0.0.1 | details=invalid_credentials | ua=Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/605.1.15 (KHTML, like Gecko) Version/26.3.1 Safari/605.1.15 | ts=2026-04-19 10:28:43

LOGIN_ATTEMPT | user=test2 | ip=127.0.0.1 | details=- | ua=Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/605.1.15 (KHTML, like Gecko) Version/26.3.1 Safari/605.1.15 | ts=2026-04-19 10:28:43

LOGIN_FAILED | user=test2 | ip=127.0.0.1 | details=invalid_credentials | ua=Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/605.1.15 (KHTML, like Gecko) Version/26.3.1 Safari/605.1.15 | ts=2026-04-19 10:28:38

LOGIN_ATTEMPT | user=test2 | ip=127.0.0.1 | details=- | ua=Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/605.1.15 (KHTML, like Gecko) Version/26.3.1 Safari/605.1.15 | ts=2026-04-19 10:28:38

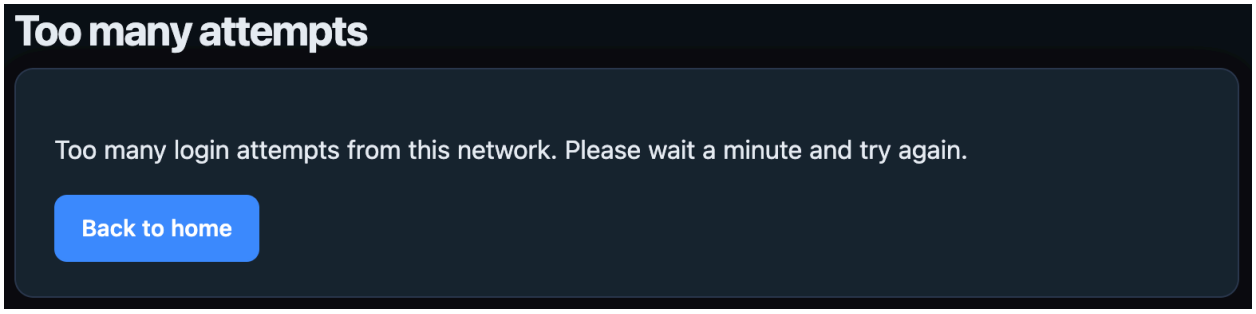
LOGIN_FAILED | user=test2 | ip=127.0.0.1 | details=invalid_credentials | ua=Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/605.1.15 (KHTML, like Gecko) Version/26.3.1 Safari/605.1.15 | ts=2026-04-19 10:28:33

LOGIN_ATTEMPT | user=test2 | ip=127.0.0.1 | details=- | ua=Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/605.1.15 (KHTML, like Gecko) Version/26.3.1 Safari/605.1.15 | ts=2026-04-19 10:28:33

LOGIN_FAILED | user=test2 | ip=127.0.0.1 | details=invalid_credentials | ua=Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/605.1.15 (KHTML, like Gecko) Version/26.3.1 Safari/605.1.15 | ts=2026-04-19 10:28:28

LOGIN_ATTEMPT | user=test2 | ip=127.0.0.1 | details=- | ua=Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/605.1.15 (KHTML, like Gecko) Version/26.3.1 Safari/605.1.15 | ts=2026-04-19 10:28:27

Rate Limiting (Multiple login attempts from the same IP is a short period of time)



SUSPICIOUS_ACTIVITY | user=- | ip=127.0.0.1 | details=Rate limit exceeded - too many login attempts from IP | ua=Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:149.0) Gecko/20100101 Firefox/149.0 | ts=2026-04-19 17:34:14

Potential Weakness (Medium):

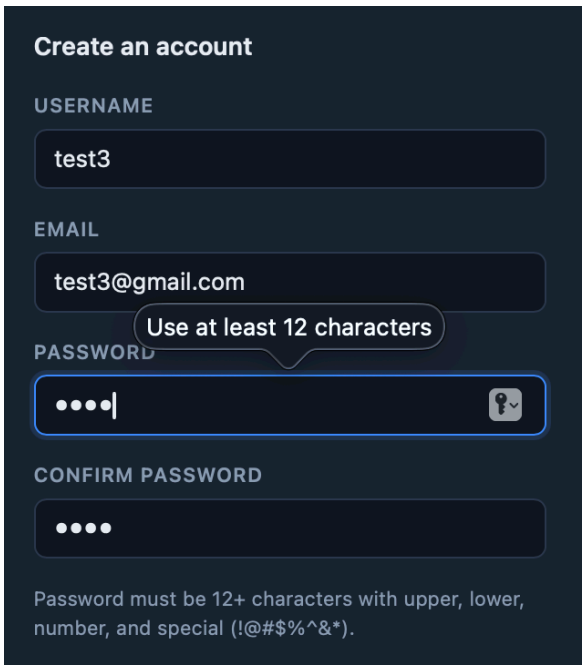
Rate limit is based on request.remote_addr without any trusted-proxy handling (like Cloudflare). This leads to attackers being able to hide behind reverse proxies to make each request seem like it came from a different IP.

Steps to Reproduce: Attack uses a VPN rotation where each login comes from a different IP address, which would not get rate limited as each IP has 10 attempts

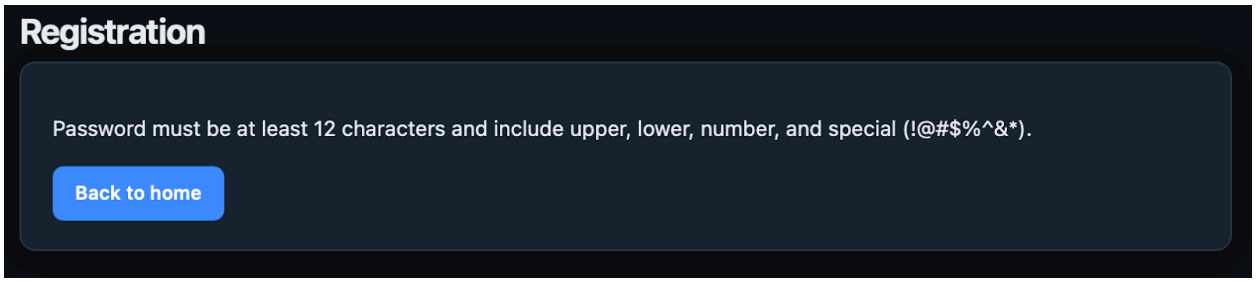
Remediation: Create a method or use a library that looks for the header which contains the original user IP when behind a proxy. Create a list of trust proxy IP addresses, otherwise fall back to direct connection IP.

- Password Complexity Enforcement

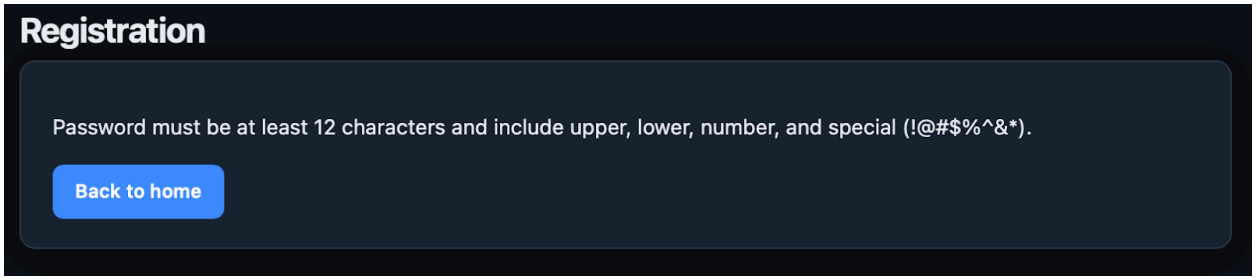
Password Attempt: “lame”



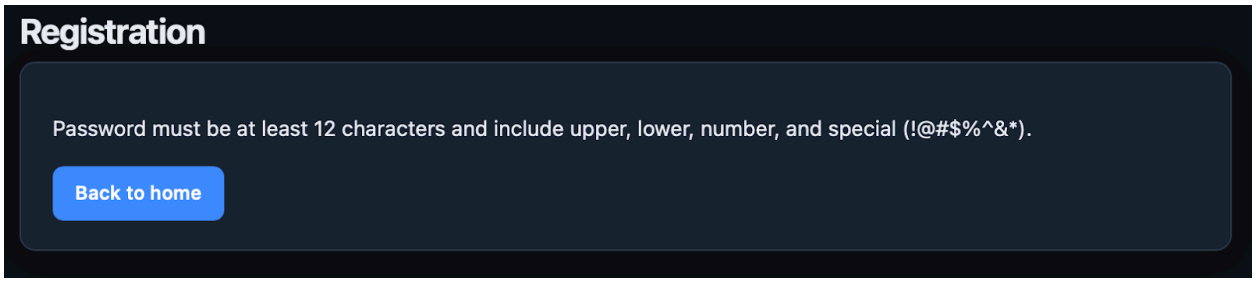
Password Attempt: “nouppercase123!”



Password Attempt: “NOLOWER123!”



Password Attempt: “NoSpecial123”



Password Attempt: “Rutgers2026!”

Note:
Input fields use .strip(), so leading/trailing whitespaces will be cut off.

<u>- Session Management</u>

Each login generates a new token and overwrites the cookie via attach_session_cookie() (Tested with Browser Developer Tools, to ensure proper session tokens)

Headers | Cookies | Request | Response | Timings | Security

Filter Cookies

Response Cookies

session_token:

expires: "2026-04-19T22:25:04.000Z"

httpOnly: true

path: "/"

samesite: "Strict"

value: "LAZsjFVHkD577fdWCi3Dv_yFQg31bWhef96D-3uxT5g"

Headers | Cookies | Request | Response | Timings | Security

Filter Cookies

Response Cookies

session_token:

expires: "2026-04-19T22:26:46.000Z"

httpOnly: true

path: "/"

samesite: "Strict"

value: "du5IpEtV2gaeDKO3fjE_t4VXRN88jfCY1mb31F5oA50"

Force Close Functionality (Admin force closes a session from Admin Panel)

FORCE_CLOSED_BY_ADMIN | user=admin | ip=127.0.0.1 | details=Z1LNMO5G39i9DAByeYneclzGH4NoYV_N0vGNw4PIzv8 | ua=Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:149.0) Gecko/20100101 Firefox/149.0 | ts=2026-04-19 10:48:56

SESSION_FORCE_CLOSED | user=admin | ip=127.0.0.1 | details=terminated_user=yat | ua=Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:149.0) Gecko/20100101 Firefox/149.0 | ts=2026-04-19 10:48:56

- Logout Functionality

Logout removes server session and expires session token (Simply logging out of the active account)

SESSION_DESTROYED | user=admin | ip=127.0.0.1 | details=- | ua=Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:149.0) Gecko/20100101 Firefox/149.0 | ts=2026-04-19 17:59:41

Headers | Cookies | Request | Response | Timings | Security

Filter Cookies

Response Cookies

```
session_token: {...}
  expires: "2026-04-19T21:59:41.000Z"
  httpOnly: true
  path: "/"
  samesite: "Strict"
  value: ""
```

Using old expired session token no longer works

Using expired session token.

SUSPICIOUS_ACTIVITY | user=- | ip=127.0.0.1 | details=Invalid session token used | ua=Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:149.0) Gecko/20100101 Firefox/149.0 | ts=2026-04-19 19:20:51

- Password Reset Security

Attempting to access the reset password route as a non-admin is forbidden (403). (Dev Tools Console Testing)

```
>> fetch("https://127.0.0.1:5001/admin/reset-password/yat", {
     method: "POST",
     headers: {
       "Content-Type": "application/json"
     },…
<- Promise { <state>: "pending" }
XHR POST https://127.0.0.1:5001/admin/reset-password/yat [HTTP/1.1 403 FORBIDDEN 5ms]
```

Forgot Password?

Forgot password

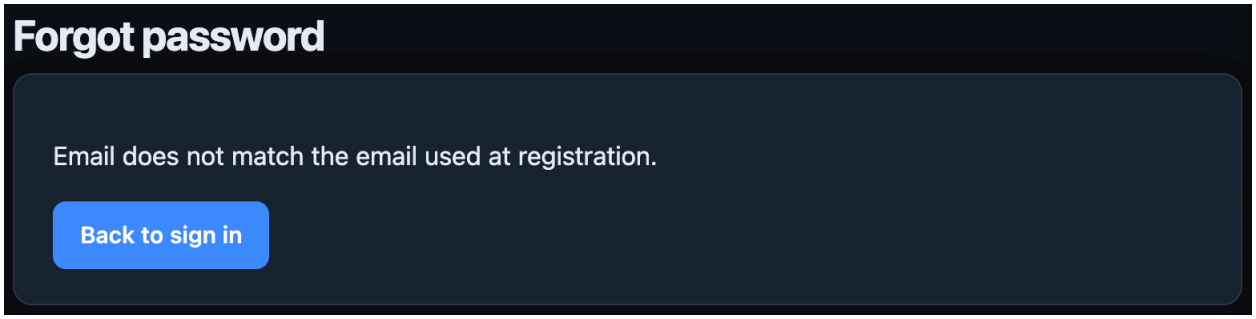
Your password reset request has been sent to admin. Once the admin has reset your password, you will receive an email with your new password.

Back to sign in

PASSWORD_RESET_REQUESTED_BY_USER | user=yat | ip=127.0.0.1 | details=yat@gmail.com | ua=Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:149.0) Gecko/20100101 Firefox/149.0 | ts=2026-04-19 22:28:07

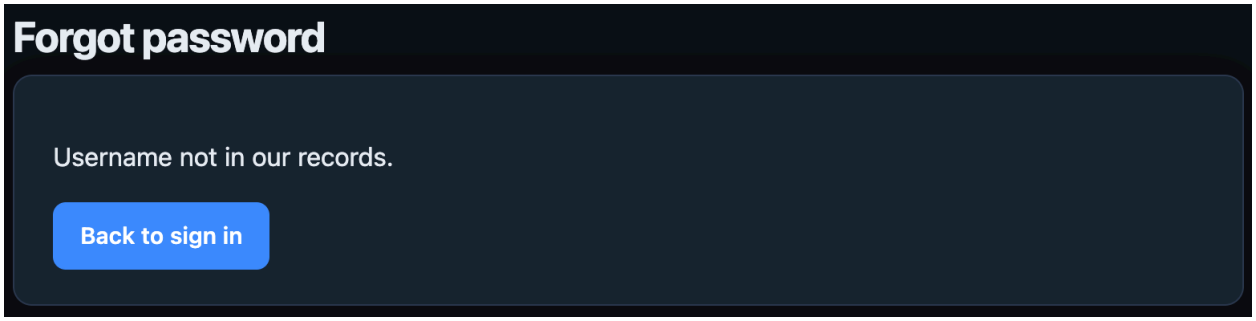
Same input validation is applied to the reset password fields.

Username and registration emails must match



PASSWORD_RESET_REQUEST_FAILED | user=yat | ip=127.0.0.1 | details=email_mismatch | ua=Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:149.0) Gecko/20100101 Firefox/149.0 | ts=2026-04-19 22:31:11

Invalid username



PASSWORD_RESET_REQUEST_FAILED | user=yatt | ip=127.0.0.1 | details=unknown_user | ua=Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:149.0) Gecko/20100101 Firefox/149.0 | ts=2026-04-19 22:32:13

Admin resets password

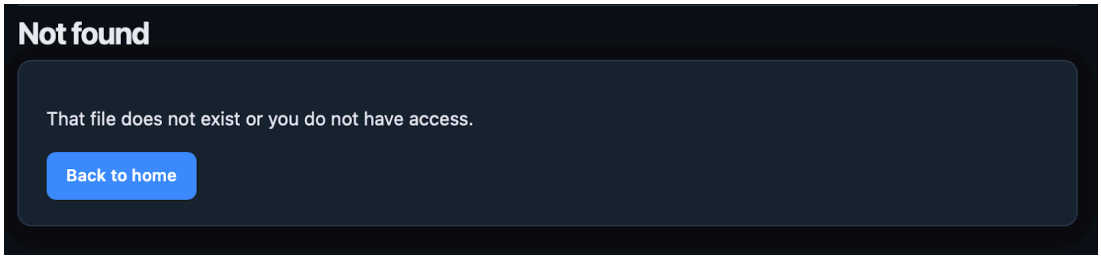
PASSWORD_RESET_BY_ADMIN | user=admin | ip=127.0.0.1 | details=yat | ua=Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/605.1.15 (KHTML, like Gecko) Version/26.4 Safari/605.1.15 | ts=2026-04-19 22:32:56

Potential Weakness (HIGH):
After resetting the password, the admin can easily login to any user account. This can be prevented by using an automated system to send out the reset password instead of having the admin manually do it. Generate a single-use token with an expiry, then email it to address on file automatically. Another way to mitigate this vulnerability is to hide the username from the admin (still can see email). But the main fix is separation of duties.

B. Authorization Testing (8 points)

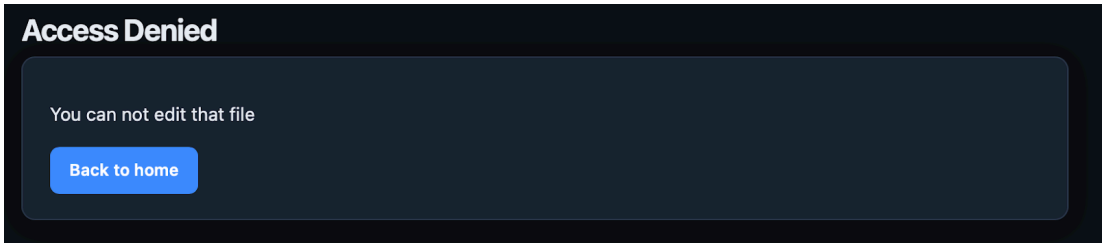
- Horizontal Privilege Escalation + Direct Object Reference

User A cannot access User B's files



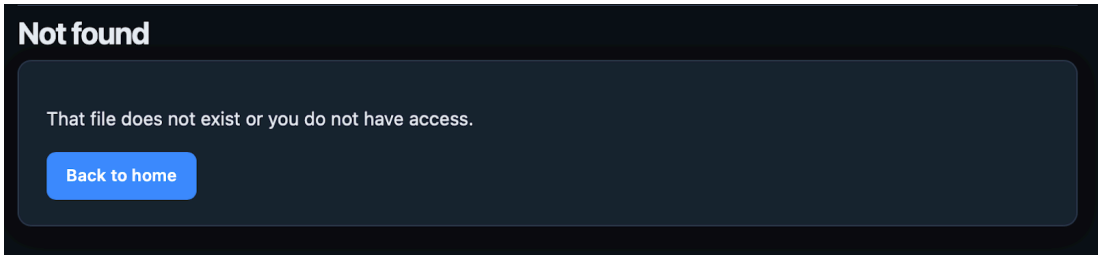
AUTHORIZATION_FAILURE | user=yat | ip=127.0.0.1 | details=Attempted to open file d07add3c-846b-43b5-964e-9eb3f4795800 | ua=Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/605.1.15 (KHTML, like Gecko) Version/26.4 Safari/605.1.15 | ts=2026-04-19 18:17:03

User A cannot edit User B's files



AUTHORIZATION_FAILURE | user=yat | ip=127.0.0.1 | details=Attempted to edit file bb465ebd-7bb1-4fc6-8dca-6dedf357c367 | ua=Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/605.1.15 (KHTML, like Gecko) Version/26.4 Safari/605.1.15 | ts=2026-04-19 18:15:43

User A cannot download User B's files



AUTHORIZATION_FAILURE | user=yat | ip=127.0.0.1 | details=Attempted to download file bb465ebd-7bb1-4fc6-8dca-6dedf357c367 | ua=Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/605.1.15 (KHTML, like Gecko) Version/26.4 Safari/605.1.15 | ts=2026-04-19 18:14:17

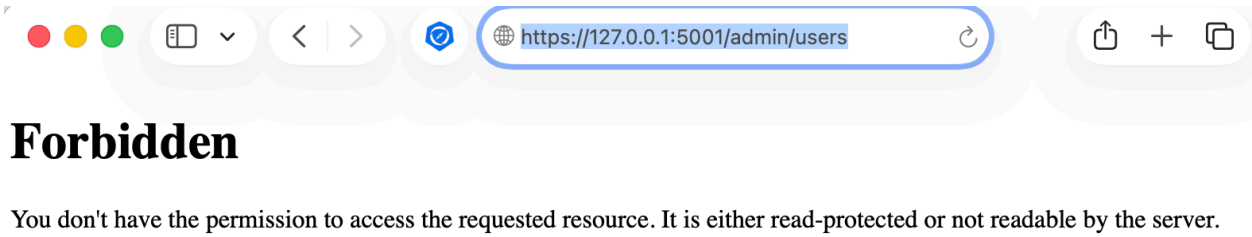
User A cannot delete User B's files "Returns to home screen"

No profile data saved, so there is no way to access another profile's personal information

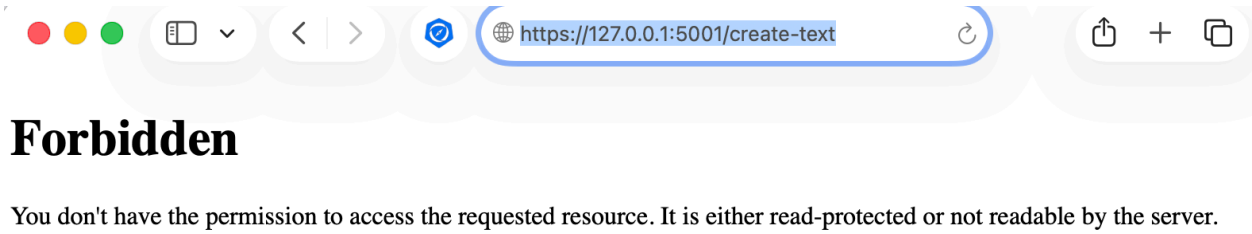
The above methods were tested with manual URL inputs while switching between accounts

- Vertical Privilege Escalation

User attempting to access admin page

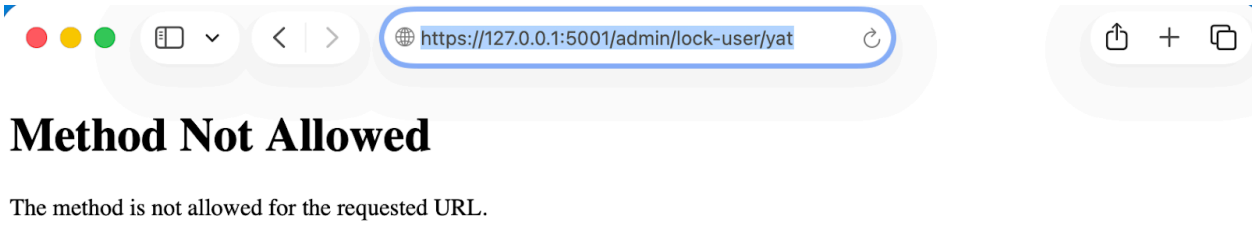


User with “guest” role attempting to access page reserved for roles “user” and “admin”



- Forced Browsing

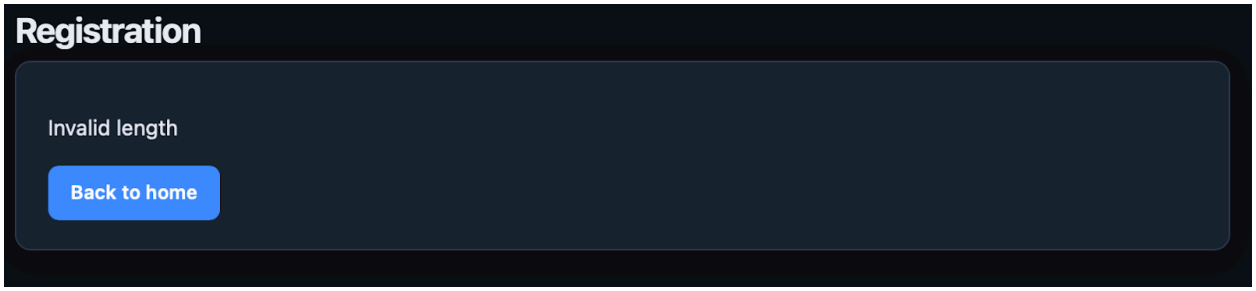
Attempting GET on POST only endpoint



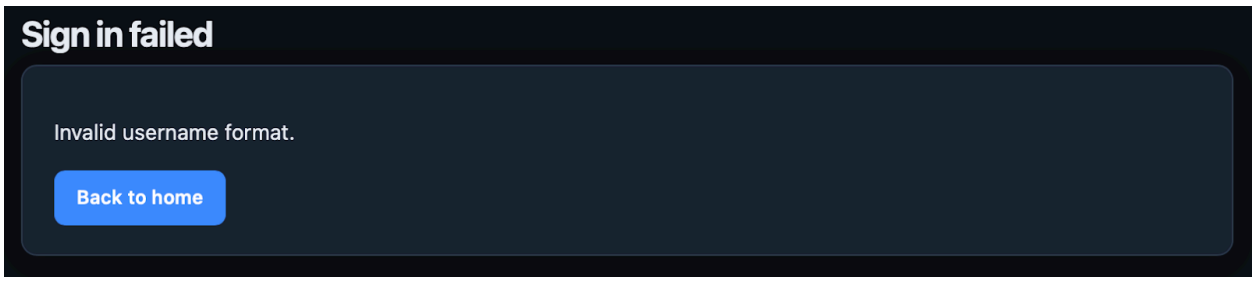
C. Input Validation Testing (10 points)

- XSS attempts

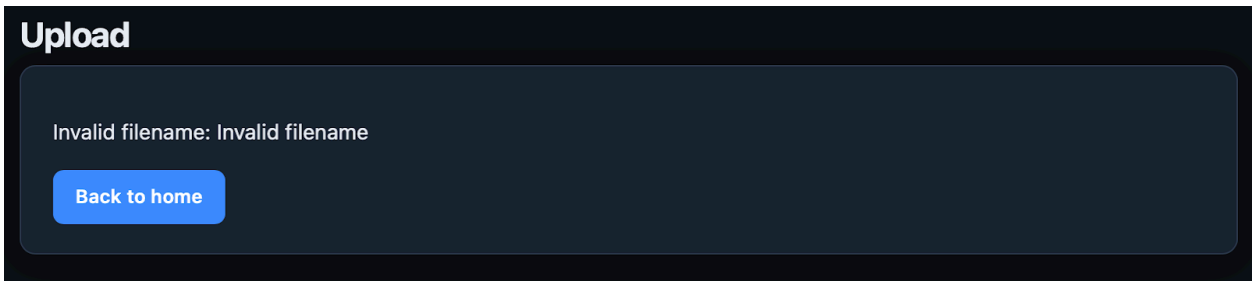
Registration Username - <script>alert('XSS')</script> (Can’t fit all due to built in 20 character limit)



Login Username - <script>alert('XSS')</script> (Can’t fit all due to built in 20 character limit)



Filename - <script>alert('XSS')</script>.txt



Infile XSS - Rendered by Jijna’s auto-escape

XSS URL Injection - No Alert Shown

Method Not Allowed

The method is not allowed for the requested URL.

- Path Traversal

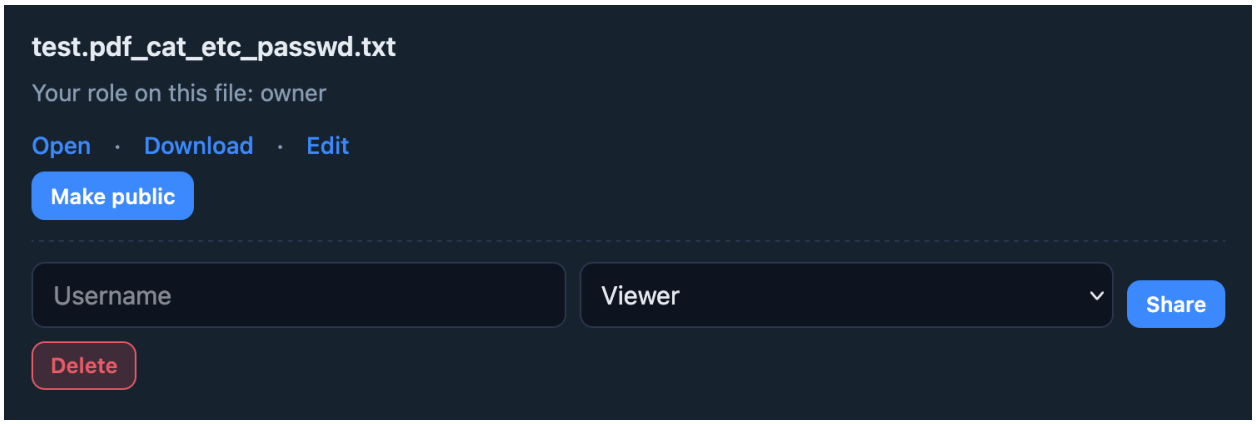
Request: https://127.0.0.1:5001/download/..%2F..%2Fetc%2Fpasswd

Not Found

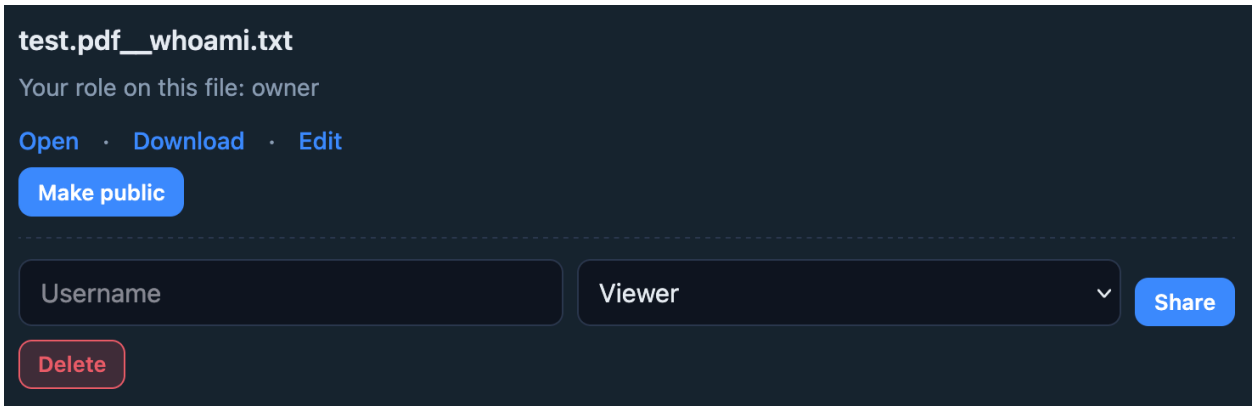
The requested URL was not found on the server. If you entered the URL manually please check your spelling and try again.

- Command injection

Create text with filename: test.pdf; cat /etc/passwd



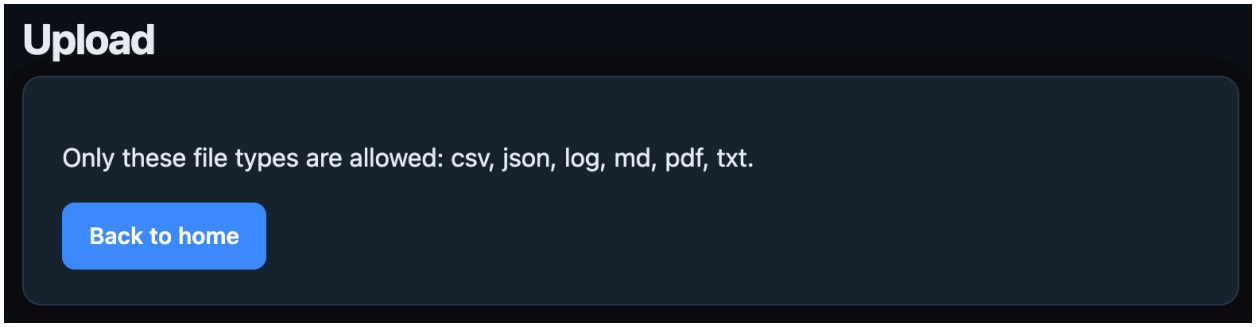
Create text with filename: test.pdf && whoami



Safely creates file with sanitized name without executing command

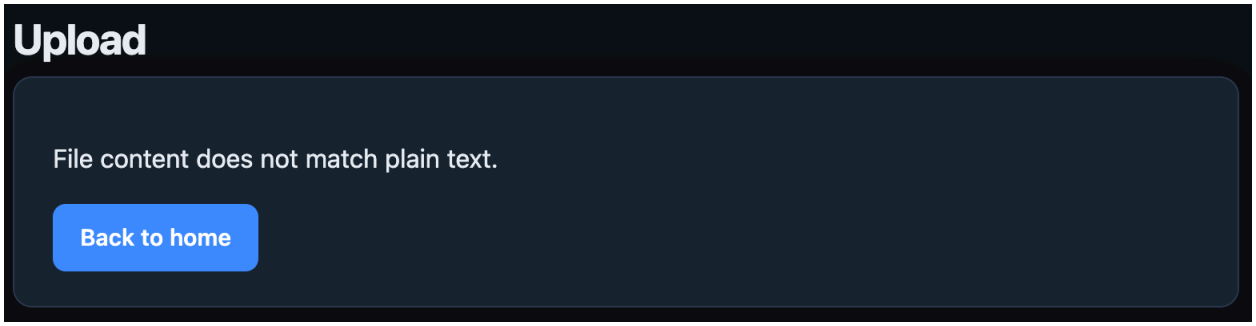
- File Upload Vulnerabilities

Attempting to upload a file not in the allowed file type list



FILE_UPLOAD_REJECTED | user=admin | ip=127.0.0.1 | details=app.py: Only these file types are allowed: csv, json, log, md, pdf, txt. | ua=Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:149.0) Gecko/20100101 Firefox/149.0 | ts=2026-04-19 11:25:52

Uploading a file that whose content does not match its file type



FILE_UPLOAD_REJECTED | user=admin | ip=127.0.0.1 | details=test.txt: File content does not match plain text. | ua=Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:149.0) Gecko/20100101 Firefox/149.0 | ts=2026-04-19 19:04:25

Done by renaming the extension of a pdf to .txt

D. Session Security Testing (7 points)

- Session Fixation

Upon login, session_token is securely generated and overwrites any existing attacker provided tokens


```
Data
▼ session_token:"attacker_chosen_value"
    Created:"Sun, 19 Apr 2026 23:15:39 GMT"
    Domain:"127.0.0.1"
    Expires / Max-Age:"Mon, 20 Apr ...23:15:29 GMT"
    HostOnly:true
    HttpOnly:false
    Last Accessed:"Sun, 19 Apr 2026 23:15:47 GMT"
    Path:"/"
    SameSite:"Lax"
    Secure:false
    Size:34
    Updated:"Sun, 19 Apr 2026 23:15:47 GMT"
```

```
Data
▼ session_token:"pH9WcbnA4gsC3...ghJ6libSSpuY"
    Created:"Sun, 19 Apr 2026 23:17:39 GMT"
    Domain:"127.0.0.1"
    Expires / Max-Age:"Sun, 19 Apr 2...23:47:39 GMT"
    HostOnly:true
    HttpOnly:true
    Last Accessed:"Sun, 19 Apr 2026 23:17:39 GMT"
    Path:"/"
    SameSite:"Strict"
    Secure:false
    Size:56
    Updated:"Sun, 19 Apr 2026 23:17:39 GMT"
```


- Session Hijacking

Obtain a valid session token from one IP/UA, then manually set the cookies in another IP/UA to use that session token.

```
DATA_CREATE | user=yat | ip=127.0.0.1 | details=Created text file: hijacked.txt | ua=Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36 | ts=2026-04-19 19:29:05

LOGIN_SUCCESS | user=yat | ip=127.0.0.1 | details=- | ua=Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:149.0) Gecko/20100101 Firefox/149.0 | ts=2026-04-19 19:27:22

SESSION_CREATED | user=yat | ip=127.0.0.1 | details=- | ua=Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:149.0) Gecko/20100101 Firefox/149.0 | ts=2026-04-19 19:27:22

LOGIN_ATTEMPT | user=yat | ip=127.0.0.1 | details=- | ua=Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:149.0) Gecko/20100101 Firefox/149.0 | ts=2026-04-19 19:27:22
```

This shows that session hijacking is possible if the attacker is able to intercept the session cookies. The security measures already taken here are using HTTPS and setting “Secure” and “HttpOnly” flags.

Potential Weakness (Medium):
Obtaining a valid session token gives the attacker the same privileges as the stolen session’s user.

Scenario/Steps to Reproduce:

1. Login to an account with user role and obtain session token from cookies in browser’s dev tools.
2. Manually set the cookies in another IP/UA to contain the stolen session token.
3. Attempt a user role route (e.g. https://127.0.0.1:5001/create-text)

Remediation: Bind sessions to IP and/or User-Agent at validate_session() in sessions.py. Then log any IP/UA mismatches as attempted session hijacking.

- Concurrent Session Handling

Users can login from different IP/UA at the same time.

```
LOGIN_SUCCESS | user=yat | ip=127.0.0.1 | details=- | ua=Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36 | ts=2026-04-19 19:37:48

SESSION_CREATED | user=yat | ip=127.0.0.1 | details=- | ua=Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36 | ts=2026-04-19 19:37:48

LOGIN_ATTEMPT | user=yat | ip=127.0.0.1 | details=- | ua=Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36 | ts=2026-04-19 19:37:48

LOGIN_SUCCESS | user=yat | ip=127.0.0.1 | details=- | ua=Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:149.0) Gecko/20100101 Firefox/149.0 | ts=2026-04-19 19:37:46

SESSION_CREATED | user=yat | ip=127.0.0.1 | details=- | ua=Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:149.0) Gecko/20100101 Firefox/149.0 | ts=2026-04-19 19:37:46

LOGIN_ATTEMPT | user=yat | ip=127.0.0.1 | details=- | ua=Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:149.0) Gecko/20100101 Firefox/149.0 | ts=2026-04-19 19:37:46
```

Force closing one session leaves the other session intact.

Demotion Functionality (user → guest triggers session clear)

```
USER_DEMOTED_TO_GUEST | user=admin | ip=127.0.0.1 | details=yat | ua=Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:149.0) Gecko/20100101 Firefox/149.0 | ts=2026-04-19 10:52:38
```

```
SESSIONS_CLEARED_FOR_USER | user=admin | ip=127.0.0.1 | details=target=yat,count=2 | ua=Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:149.0) Gecko/20100101 Firefox/149.0 | ts=2026-04-19 10:52:38
```

Potential Weakness (Low):
There is no concurrent session management protection against race conditions.

Scenario/Steps to Reproduce:

1. Two concurrent sessions call validate_session()
2. Request 1 loads sessions from storage: sessions_1
3. Request 2 loads sessions from storage: sessions_2
4. Request 1 validates and updates sessions and saves to storage: sessions_1_updated
5. Request 2 validates and updates sessions and saves to storage: sessions_2_updated
6. sessions_2_updated is saved in storage → changes from request 1 is lost

Remediation: Implement file locking when reading and writing. Implement a request system that would have a queue like system where if a lock is already held, the second request would have to wait until the first one finishes.

- Session Timeout

Session Timeout Functionality (1800 seconds)

```
SESSION_TIMEOUT | user=yat | ip=127.0.0.1 | details=idle_seconds=1892 | ua=Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:149.0) Gecko/20100101 Firefox/149.0 | ts=2026-04-19 11:41:54
```

E. Configuration Testing (5 points)

- Security Headers

(Using Browser Developer Tools, we can check on the response from HTML, with our headers)

```
Response
HTTP/1.1 200 OK
Connection: close
Content-Length: 2934
Content-Security-Policy: default-src 'self'; script-src 'self' 'unsafe-inline'; style-src 'self' 'unsafe-inline'; img-src 'self' data:; font-
    src 'self'; connect-src 'self'; frame-ancestors 'none'
Content-Type: text/html; charset=utf-8
Date: Sun, 19 Apr 2026 23:39:13 GMT
Permissions-Policy: geolocation=(), microphone=(), camera=()
Referrer-Policy: strict-origin-when-cross-origin
Server: Werkzeug/2.2.2 Python/3.10.7
Strict-Transport-Security: max-age=31536000; includeSubDomains
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
X-XSS-Protection: 1; mode=block
```

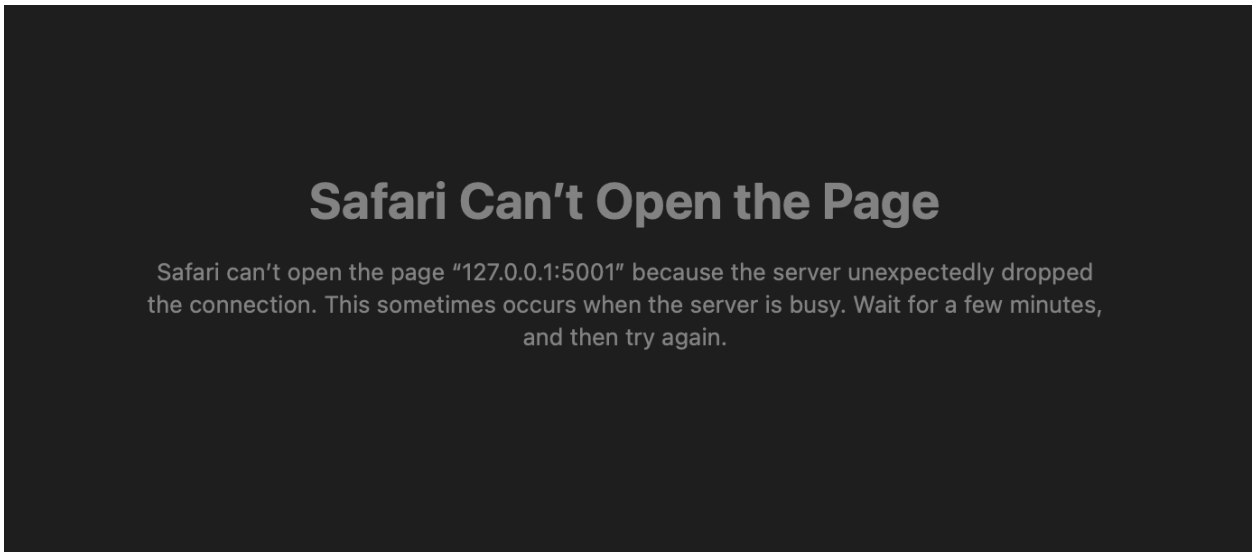
Potential Weakness (Low):

The server header reveals the framework and version information when it says “Werkzeug/2.2.2 Python/3.10.7”. This can lead to an attack to target any known vulnerabilities for those specific versions. Steps to Reproduce: Load page, inspect element in browser, open network tab, and click on HTML where you will then see the server response with the version information.

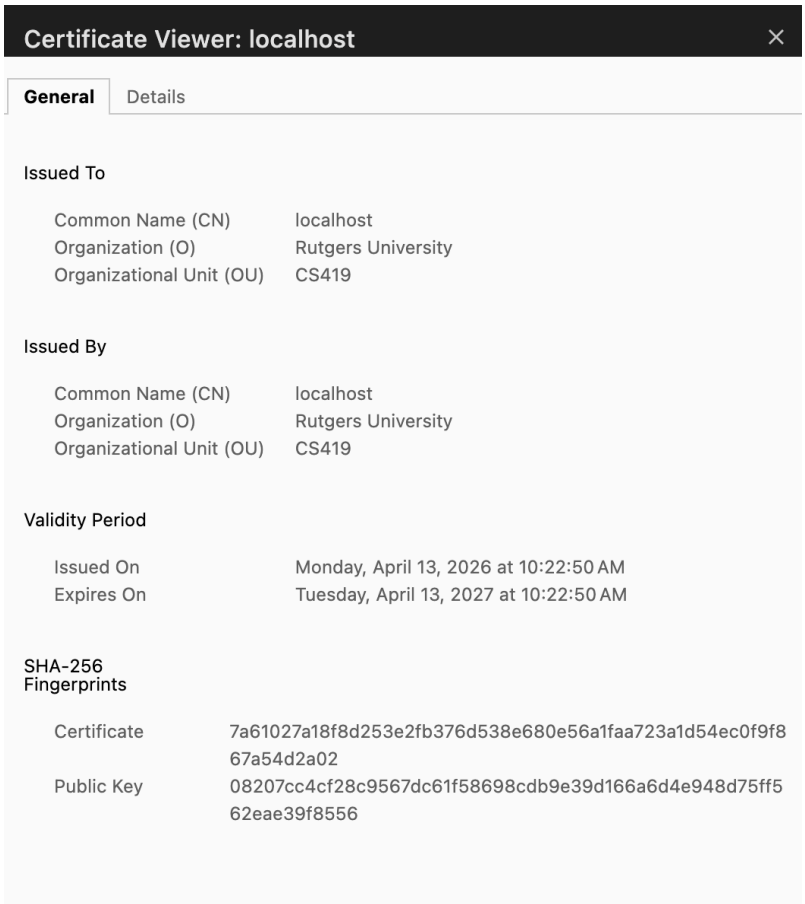
Remediation: Remove the server header from all HTTP responses. Otherwise you can override the header to say the same information just without the version numbers.

- TLS/SSL configuration

HTTPS Enforced (When attempting to http instead of https with https://127.0.0.1:5001/, since HTTP would need to be hosted on another server too we have it show as a “Failed to open page”



Valid Certificate (Tested using Browser Developer Tools, to view active certificate with expiration date and subject)



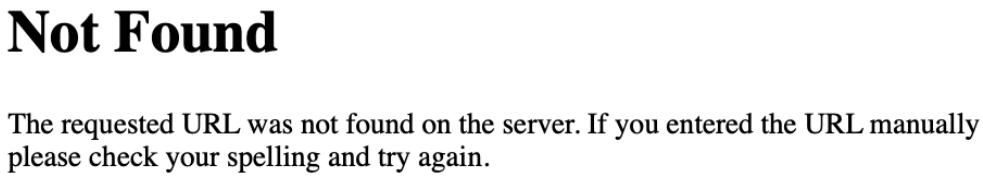
TLS Protocol Version (Found using browser development tools, such as inspect element and network settings)

SSL Blocked (Command line prompt to ensure SSL is blocked)

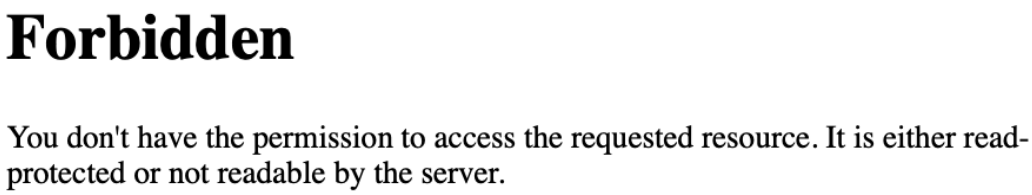
```
krishanthbabu@Krishs-Mac CompSec-Project % openssl s_client -connect 127.0.
0.1:5001 -ssl3
s_client: Unknown option: -ssl3
```

- Error handling

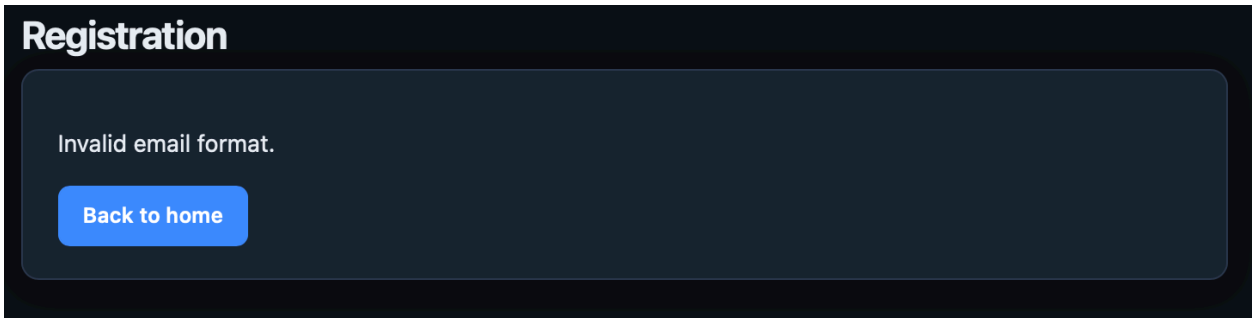
404 Not Found Error - (Tested with non-existent path https://127.0.0.1:5001/this-page-does-not-exist)



403 Forbidden Error - (Tested with https://127.0.0.1:5001/admin/users while logged in as a regular user)



Invalid Email Format - (Tested with test5@g)



- Debug mode disabled

Debug = False

```
* Serving Flask app 'app'
* Debug mode: off
```

Conclusion

Our Secure Document Sharing Program was able to successfully meet all security requirements that were outlined in our project specifications. During our testing through the penetration report, we were able to outline a few small areas of concern, but at the same time build confidence in the working pieces of our program. After a full test, we were able to find 0 Critical, 1 High, 1 Medium, and 2 Low-severity issues. All findings were documented with proof, alongside potential weaknesses, steps to reproduce and remediation tips for any issues. Based on our findings we believe that our program would be approved for public deployment after some final changes.